

Project 1: 文本无损信源编码传输

曾闻天 231880147

1 实验目标

在信源编码理论中，Huffman 编码是一种最优前缀编码方法，其核心思想是：为出现频率高的符号分配较短的码字，为频率低的符号分配较长的码字，从而使得平均码长最短。

项目一的目标是实现一个基于 Huffman 编码的文本无损信源编码系统，并验证其编码效率。

2 实验过程

2.1 读取文本信源

函数：`load_text(path: Path) -> str`

优先以 UTF-8 编码读取文本文件，若失败则尝试使用 GBK 编码读取，作为原始信源符号序列。

2.2 统计符号频率

函数：`Counter(text)`

将 2.1 中读取到的文本视为字符序列，统计每个字符出现的频次，作为信源概率分布的估计。

2.3 构建 Huffman 树

函数：`build_huffman_tree(freq: Counter) -> HuffmanNode`

构建过程分为三个阶段。首先是边界处理：如果频率表为空，直接抛出异常；如果只有一个符号，则直接返回一个叶子节点。

由于这种单节点树既是根也是叶子，后续的 `build_codes` 函数中需要额外处理以正确返回编码。

第二阶段是初始化堆。程序遍历频率表，为每个字符创建一个 HuffmanNode 叶子节点，然后将节点包装成 HeapItem(freq, tie, node) 的形式压入最小堆。其中的 tie 从 0 开始递增，用来确保相同频率的节点在堆中有确定的顺序。

最后是迭代合并。只要堆中元素数量大于 1，就弹出两个最小的节点，创建一个新的内部节点，其频率为两者之和，左右子节点分别为这两个节点。新节点会被压回堆中，同时 tie 继续递增。重复这一过程，直到堆中只剩下一个节点，它就是整棵 Huffman 树的根。

2.4 生成 Huffman 码表

函数：build_codes(root: HuffmanNode) -> Dict[str, str]

构建码表的过程采用经典的 DFS 递归遍历。

当到达一个叶子节点时，将当前路径 prefix 记录为该字符的 Huffman 码字。

前文提到了这里有一个特殊情况：如果整棵树只有一个叶子节点，即输入文本只包含一种字符时，那么递归过程中 prefix 会一直保持为空字符串。由于码字不能为空，此时需要手动赋值为 0。对于内部节点，则继续递归遍历：向左走时在路径末尾追加 0，向右走时追加 1。

2.5 编码（压缩）

函数：HuffmanCoder.encode(text: str) -> bytes

首先，将文本中的每个字符替换为对应的 Huffman 码字，得到一个由 0 和 1 组成的二进制字符串。由于每个字节有 8 位，需要调用 bits_to_bytes 函数将这个二进制字符串补齐到 8 的倍数，再打包成字节序列。

同时，为了能够正确解压，还需要生成一个 JSON 格式的文件头，其中包含两个关键信息：bit_count 表示有效比特数（用于去除补齐时添加的多余位），以及 codes 记录每个字符的 Huffman 码表。

最终输出的文件格式为：先是 4 字节的魔数 HUF1，紧接着 4 字节的头长度，然后是 JSON 头，最后是压缩后的数据本体。

2.6 解码（解压）

函数：HuffmanCoder.decode(data: bytes) -> str

解压时，首先解析文件开头的魔数和 JSON 头，从中恢复出 Huffman 码表以及有效比特数。然后调用 bytes_to_bits 函数将后续的载荷数据还原为原始的二进制字符串。

接着遍历这个比特流，利用码表进行反向匹配——由于 Huffman 码是前缀码，每个码字都可以被唯一解码，因此可以逐个还原出原始字符。

为了确保压缩过程没有造成信息丢失，系统还实现了无损验证机制：在编码完成后立即调用 `decode` 函数对压缩结果进行解压，并将解压结果与原始文本进行比较。这一过程封装在 `verify_lossless` 函数中。

2.7 效率分析

为了评估 Huffman 编码的压缩效果，系统提供了一系列分析指标。

首先是 Shannon 熵，通过函数 `shannon_entropy(freq)` 计算，

$$H = - \sum p_i \log_2 p_i \text{ (比特/符号)},$$

其中 $p_i = \text{count}_i / \text{总字符数}$ ，它表示信息论下的理论最优平均码长。

接着计算平均码长，由 `average_code_length(freq, codes)` 实现，

$$L = \sum p_i \cdot \text{len}(\text{code}_i) \text{ (比特/符号)},$$

这是实际编码的平均长度。

由此可以导出两个核心指标：

效率 $\eta = H/L$ （越接近 1 表示编码越优）和冗余 $L - H$ （每个符号浪费的比特数）。

在压缩比分析方面，原始字节数为 `len(text.encode("utf-8"))`，压缩后字节数为 `len(compressed)`，压缩比定义为原始字节除以压缩字节。还可以计算实际比特/符号，即 $(\text{压缩字节} \times 8) / \text{字符数}$ ，便于与理论熵值对比。

作为参考，系统会使用 `zlib.compress(level=9)` 的输出作为通用无损压缩算法的对比基准，从而衡量 Huffman 编码在实际文本上的相对表现。

2.8 报告输出

函数：`print_report(text, compressed, path)`

输出内容包括：

指标	说明
符号数、唯一符号数	文本长度与字符种类
Shannon 熵 H	理论最优平均码长下限
Huffman 平均码长 L	实际平均码长
效率 η	H/L
冗余度	$L - H$
压缩比	原始 UTF-8 字节 / Huffman 压缩字节
zlib 参考压缩比	对比通用压缩器
无损验证	解码结果是否与原文一致

3 交互

交互菜单 (interactive_loop)

支持三种操作：

1. 编码并打印报告（不保存文件）
2. 编码并保存 .huf 文件
3. 解码 .huf 并还原为 .txt

4 实验结果

1. 运行 pro1.bat，选择对 test1.txt 与 test2.txt [1] 编码并分析。

2. 程序输出:

```
=====
信源编码效率报告 - E:\NJU-ISE-Info Theory\pro1\test1.txt
=====
符号数 (字符)           : 5675
不同符号种类数         : 693
Shannon 熵 H (每符号)   : 8.1579 bit/符号
Shannon 熵 H (每字节)   : 5.8294 bit/字节 (UTF-8)
Huffman 平均码长 L      : 8.1914 bit/符号
冗余度 (L - H)          : 0.0335 bit/符号
编码效率  $\eta = H/L$     : 0.9959 ( $\leq 1$ , 接近 Kraft 界)
=====
原文体积 (UTF-8)        : 15969 字节
压缩后体积 (Huffman)    : 20644 字节
压缩比 (原/压)          : 0.7735
实际比特/符号 (含开销)  : 29.1017
zlib 参考               : 6180 字节 (压缩比 2.5840)
=====
无损往返验证: 通过
```

图 1: test1

```
=====
信源编码效率报告 - E:\NJU-ISE-Info Theory\pro1\test2.txt
=====
符号数 (字符)           : 5122236
不同符号种类数         : 5062
Shannon 熵 H (每符号)   : 8.7812 bit/符号
Shannon 熵 H (每字节)   : 5.7412 bit/字节 (UTF-8)
Huffman 平均码长 L      : 8.8090 bit/符号
冗余度 (L - H)          : 0.0277 bit/符号
编码效率  $\eta = H/L$     : 0.9969 ( $\leq 1$ , 接近 Kraft 界)
=====
原文体积 (UTF-8)        : 14826692 字节
压缩后体积 (Huffman)    : 5778685 字节
压缩比 (原/压)          : 2.5658
实际比特/符号 (含开销)  : 9.0253
zlib 参考               : 6059630 字节 (压缩比 2.4468)
=====
无损往返验证: 通过
```

图 2: test2

3. 结果分析

综合以上测试, 可以得出如下结论。

第一, 无损性验证通过: 两个信源在编码后立即解码并与原文比较, 结果完全一致, 编解码过程正确无误。

第二, 编码效率 η 均约为 0.996, 说明 Huffman 树的实现正确且接近 Kraft 不等式所给出的最优边界。

第三，从体积压缩角度看，短文本（test1）受 JSON 码本头部开销主导，压缩比小于 1 属于预期行为；长文本（test2）则充分体现了 Huffman 编码对重复长文本的实际压缩能力。

5 总结

通过上述实验，项目一的研究目标基本实现。